Projekt I przetwarzanie równoległe 2025/26 SS, OpenMP, Vtune

Emilia Bonk 160108
Szymon Urban 160271

Wstęp.

- **Nazwa przedmiotu:** Laboratorium z przetwarzania równoległego
- **Autorzy:** Emilia Bonk (160108), Szymon Urban (160271)
- **Grupa dziekańska:** L09
- **Termin zajęć laboratoryjnych:** Wtorek 9:45-11:15
- **Termin oddania sprawozdania:** 5.05.2026
- **Wersja sprawozdania:** pierwsza
- **Opis zadania:** Celem ćwiczenia była analiza implementacji algorytmu Sita Eratostenesa w środowisku równoległym i sekwencyjnym z wykorzystaniem biblioteki OpenMP. Zadanie polegało na analizie i modyfikacji podanych kodów, z róznymi implementacjami algorytmu, a następnie porównaniu ich wydajności.
- **Adres e-mail:** emilia.bonk@student.put.poznan.pl, szymon.urban@student.put.poznan.pl

1 Informacje techniczne.

Parametr	**Wartość**
Model procesora	17th Gen Intel(R) Core(TM) i7-12700F
Liczba procesorów fizycznych	1
Liczba rdzeni	12
Liczba procesorów logicznych	20
Pamięć cache L1	1,0 MB
Pamięć cache L2	12,0 MB
Pamięć cache L3	25,0 MB
System operacyjny	Windows 10
Kompilator	Intel(R) oneAPI DPC++/C++ Compiler 2025.3.3
Wersja kompilatora	2025.3.3.20260319
Program testujący	Intel VTune Profiler 2026.0.0

Tablica 1: Informacje techniczne dotyczące środowiska testowego

2 Prezentacja i analiza kodów.

K1 - Algorytm sekwencyjny oparty na próbnych dzieleniach.

Oznaczenie skrótowe: SEQ-DIV-BASIC

Wariant oznaczony jako SEQ-DIV-BASIC to algorytm sekwencyjny oparty na metodzie dzielenia próbnego. Przetwarzanie rozpoczyna się od przygotowania struktur danych: tablicy result dla zakresu $[m, n]$ oraz pomocniczej tablicy primeArray przechowującej liczby pierwsze z przedziału $[2, \sqrt{n}]$. Pierwsza pętla for wypełnia tablicę pomocniczą, eliminując z rozważania liczby złożone, natomiast druga pętla stanowi główne zadanie obliczeniowe, sprawdzając podzielność każdej liczby i przez wyznaczone wcześniej dzielniki. W tym podejściu zbiór zadań obejmuje weryfikację $n - m + 1$ elementów, przy czym koszt wykonania pojedynczego zadania jest wysoce zmienny. Liczby parzyste lub posiadające małe dzielniki są procesowane błyskawicznie dzięki instrukcji break, podczas gdy liczby pierwsze wymagają wykonania pełnego cyklu sprawdzeń aż do wartości $\sqrt{i}$, co sprawia, że czas obliczeń rośnie wraz z wartością sprawdzanej liczby. Ponieważ jest to kod sekwencyjny, nie występuje tu podział pracy między jednostki obliczeniowe – wszystkie zadania są przydzielane do jednego wątku głównego i wykonywane szeregowo. Zastosowanie tablic boolowskich i zmiennych licznikowych i oraz j pozwala na bezpośrednią iterację po zadanym zbiorze, zapewniając pełną deterministyczność przebiegu algorytmu i dając zawsze poprawny wynik.

```
int calculate(int n, int m){
        bool* result = (bool*)malloc((n - m + 1) * sizeof(bool));
        memset(result, true, (n - m + 1) * sizeof(bool));
        int prime_counter = 0;

        bool* primeArray = (bool*)malloc((sqrt(n) + 1) * sizeof(bool));
        memset(primeArray, true, (sqrt(n) + 1) * sizeof(bool));

        if (m <= 0 && n >= 0) result[0 - m] = false;
        if (m <= 1 && n >= 1) result[1 - m] = false;

        for (int i = 2; i * i <= n; i++) {
                for (int j = 2; j * j <= i; j++) {
                        if (primeArray[j] == true && i % j == 0) {
                                primeArray[i] = false;
                                break;
                        }
                }
        }
        for (int i = m; i <= n; i++){
                for (int j = 2; j * j <= i; j++){
                        if (primeArray[j] == true && i % j == 0) {
                                result[i - m] = false;
                                break;
                        }
                }
                if(result[i-m] == true){
                        prime_counter++;
                }
        }
        free(result);
        free(primeArray);

        return prime_counter;
}
```

Listing 1: Kod programu k1.cpp

K2 - Liczby pierwsze wyznaczane równolegle przez dzielenie w zakresie <m,n>.

Oznaczenie skrótowe: PAR-DIV-GUIDED

Wariant ten wprowadza zrównoleglenie głównej pętli obliczeniowej przy użyciu dyrektywy `#pragma omp parallel for schedule(guided)`. Przetwarzanie polega na dynamicznym pobieraniu porcji zadań z puli wszystkich iteracji $[m, n]$. Charakterystyczną cechą tego podejścia jest to, że rozmiar przydzielanych paczek (chunks) maleje wykładniczo w miarę postępu obliczeń. Na początku wątki otrzymują duże bloki liczb, co minimalizuje narzut związany z synchronizacją (pobieraniem zadań). W miarę zbliżania się do końca zakresu, porcje stają się coraz mniejsze, co pozwala na precyzyjne „wypełnienie" czasu pracy wszystkich rdzeni. Zbiór zadań pozostaje ten sam ($n - m + 1$ liczb), jednak dzięki temu mechanizmowi, różnice w koszcie sprawdzania małych i dużych liczb są niwelowane: wątek, który szybciej upora się ze swoją porcją, natychmiast pobiera kolejną, mniejszą partię. Zapobiega to sytuacji, w której jeden wątek wciąż pracuje nad ciężkim końcem zakresu, podczas gdy inne pozostają bezczynne. W programie nie będzie występowało zjawisko wyścigu, gdyż każdy wątek operuje na odrębnych elementach tablicy result, Zjawisko **false sharingu** może wystąpić, kolizje w pamięci cache będą miały miejsce głównie na "stykach" bloków danych tablicy, szczególnie w późniejszych etapach, gdy paczki stają się mniejsze. **Synchronizacja** jest wprowadzana przy pomocy klauzuli schedule(guided), na podstawie dostępu do wspólnego licznika iteracji. Nie ma w tej wersji kodu problemu braku **zrównoważenia obciążenia**, Mechanizm samoregulacji rozmiaru paczek sprawia, że procesory są obciążone niemal identycznie przez cały czas trwania programu.

Uzasadnienie wyboru podziału pracy: Przewagą strategii `guided` nad `static` jest rozwiązanie problemu bezczynności procesorów na końcu zakresu, gdzie liczby są najtrudniejsze do sprawdzenia. `Guided` zmniejsza narzut komunikacyjny, biorąc najpierw duże porcje (tam gdzie liczby są małe i szybkie do sprawdzenia), a następnie stopniowo przechodząc do mniejszych, co pozwala na lepsze wykorzystanie zasobów niż `dynamic`, które co prawda pozwala wybrać dowolny rozmiar paczki, ale wciąż nie jest tak efektywne jak `guided` w kontekście tego algorytmu.

```
int calculate(int n, int m){
        //...liczenie sita (PrimeArray) jak w kodzie k1

        #pragma omp parallel
        {
        #pragma omp for schedule(guided) reduction(+:prime_counter)
        for (int i = m; i <= n; i++){
                for (int j = 2; j * j <= i; j++){
                        if (primeArray[j] == true && i % j == 0) {
                                result[i - m] = false;
                                break;
                        }
                }
                if(result[i-m] == true){
                        prime_counter++;
                }
        }
        }
        free(result);
        free(primeArray);

        return prime_counter;
}
```

Listing 2: Kod programu k2.cpp

K3 - Sito sekwencyjne bez lokalności dostępu do danych.

Oznaczenie skrótowe: SEQ-SIEVE-NL

W tym wariancie zamiast sprawdzać każdą liczbę z osobna pod kątem jej dzielników, algorytm iteruje po kolejnych liczbach pierwszych i „wykreśla" (oznacza jako false) wszystkie ich wielokrotności w tablicy wyników. Przetwarzanie składa się z dwóch faz: wyznaczenia małego sita dla dzielników oraz głównego sita dla zakresu $[m, n]$. Zadaniem jest wykreślenie wielokrotności dla każdej liczby pierwszej $i \leq \sqrt{n}$. Koszt wykonania zadania jest zależny od wartości i – dla małych liczb pierwszych (np. 2, 3, 5) liczba wykreśleń jest ogromna (dla $i = 2$ jest to połowa zakresu), natomiast dla większych liczb pierwszych koszt ten gwałtownie spada. Łączna złożoność jest znacznie niższa niż w dzieleniu próbnym i wynosi $O(n \log \log n)$. Wszystkie operacje wykonywane są przez jeden wątek główny. Wykreślanie odbywa się skokowo (instrukcja j += i), co jest bardzo efektywne dla procesora (dostęp sekwencyjny z krokiem). Jako że jest to kod wykonywany sekwencyjnie, nie będzie tu żadnego z problemów związanych z równoleglizacją zadań tj false sharingu, wyścigu czy braku zrównoważenia podziału pracy.

```
int calculate(int n, int m){
        //...liczenie sita jak we wczesniejszych kodach

        for (int i = 2; i*i <= n; i++){
                if (primeArray[i]) {
                        int firstMultiple = (m / i);
                        if (firstMultiple <= 1) {
                                firstMultiple = i + i;
                        } else if (m % i) {
                                firstMultiple = (firstMultiple * i) + i;
                        } else {
                                firstMultiple = (firstMultiple * i);
                        }
                        for (int j = firstMultiple; j <= n; j+=i) {
                                result[j-m] = false;
                        }
                }
        }
        if (m <= 0 && n >= 0) result[0 - m] = false;
    if (m <= 1 && n >= 1) result[1 - m] = false;

        for (int i = m; i <= n; i++) {
        if (result[i - m]) prime_counter++;
        }

        free(result);
    free(primeArray);

        return prime_counter;
}
```

Listing 3: Kod programu k3.cpp

K3a - Sito sekwencyjne z lokalnością dostępu do danych.

Oznaczenie skrótowe: SEQ-SIEVE-LL

TO DO!!!!!

```
int calculate(int n, int m){
    //...alokacja pamieci i liczenie PrimeArray tak samo jak w kodzie k3

    int blockSize = 65536;
    int numberOfBlocks = (n - m) / blockSize;
    if ((n - m) % blockSize != 0) {
        numberOfBlocks++;
    }

    for (int i = 0; i < numberOfBlocks; i++) {
        int low = m + i * blockSize;
        int high = low + blockSize - 1;
        if (high > n) {
            high = n;
        }

        for (int j = 2; j * j <= high; j++) {
            if (primeArray[j] == true) {
                int firstMultiple = (low / j);
                if (firstMultiple <= 1) {
                    firstMultiple = j + j;
                } else if (low % j) {
                    firstMultiple = (firstMultiple * j) + j;
                }  else {
                    firstMultiple = (firstMultiple * j);
                }
                for (int k = firstMultiple; k <= high; k += j) {
                    result[k - m] = false;
                }
            }
        }
    }

        //...zliczanie prime_count i zwalnianie pamieci jak w kodzie k3
}
```

Listing 4: Kod programu k3a.cpp

K4 - Sito równoległe funkcyjne bez lokalności dostępu do danych.

Oznaczenie skrótowe: PAR-SIEVE-NL

Jest to równoległa wersja kodu SEQ-SIEVE-NL, podział pracy odbywa się na poziomie pętli wybierającej liczby pierwsze (i) za pomocą dyrektywy `#pragma omp parallel for schedule(dynamic, 1)`. Każdy wątek otrzymuje pewną pulę liczb pierwszych i odpowiada za wykreślenie wszystkich ich wielokrotności w całej tablicy result. Zadaniem wątku jest przeprowadzenie skokowej eliminacji liczb złożonych dla przypisanych mu wartości i. Koszt zadania jest wyjątkowo nierównomierny i malejący: liczba i=2 wymaga wykreślenia $N/2$ elementów, podczas gdy liczby pierwsze bliskie $\sqrt{n}$ wymagają zaledwie kilku operacji. W programie będzie występował problem **race condition**, jednak dla nas "bezpieczny" logicznie. Wynika on z faktu, że różne liczby pierwsze mają wspólne wielokrotności (np. dla $i = 2$ i $i = 3$ wspólną wielokrotnością jest 6, 12, 18...). Oba wątki będą próbowały zapisać result[j-m] = false pod ten sam indeks w tym samym czasie. Wynik końcowy będzie poprawny (bo oba wątki wpisują tę samą wartość: false), ale na poziomie architektury procesora powoduje to ogromne straty wydajności. Występować będzie również zjawisko **false sharingu**, wątki będą zapisywać często i skakać po wartościach w tablicy result, przez co często będą trafiać w te same linie cache, zmuszając system do ciągłego odświeżania danych między rdzeniami, co znacząco obniży wydajność. Problem **braku zrównoważenia obciążenia** będzie zniwelowany przez zastosowanie strategii `dynamic`, która pozwala na dynamiczne przydzielanie zadań w miarę ich wykonania, co sprawia, że wątki, które szybciej uporały się z ciężkimi zadaniami (duże i), mogą natychmiast przejąć kolejne, mniejsze zadania (małe i), zapewniając lepsze wykorzystanie zasobów. **Synchronizacja** wątków odbędzie się na końcu pętli for, gdy wszystkie wątki zakończą swoje zadania, nie ma więc problemu z synchronizacją w trakcie przetwarzania, która dodatkowo zwolniłaby działanie programu.

Uzasadnienie wyboru podziału pracy: Strategia `(dynamic, 1)` ma swoją przewagę nad `static` dzięki dynamicznemu przydzialaniu zadań wątkom (pierwsze zadania będą największe, potem kolejne są coraz mniejsze). Ze względu na tą kolejność zadań (od najcięższych na początku do najlżejszych na końcu) zdecydowaliśmy się też, nie używać strategii `guided`, która lepiej się nadaje na odwrotną tendencję trudności zadań.

```
int calculate(int n, int m){
        //...alokacja pamieci i lizcenie PrimeArray jak wczesniej

        int limit = sqrt(n);
        #pragma omp parallel for schedule(dynamic, 1)
        for (int i = 2; i <= limit; i++){
                if (primeArray[i]) {
                        int firstMultiple = (m / i);
                        if (firstMultiple <= 1) {
                                firstMultiple = i + i;
                        } else if (m % i){
                                firstMultiple = (firstMultiple * i) + i;
                        } else {
                                firstMultiple = (firstMultiple * i);
                        }
                        for (int j = firstMultiple; j <= n; j+=i) {
                                result[j-m] = false;
                        }
                }
        }

        //zliczanie prime_count i zwalnianie pamieci jak wczesniej
}
```

Listing 5: Kod programu k4.cpp

K4a - Sito równoległe funkcyjne bez lokalności dostępu do danych z usunięciem redundancji zapisów.

Oznaczenie skrótowe: PAR-SIEVE-NL-A

Jest to wersja programu PAR-SIEVE-NL z dodatkiem sprawdzania warunku `if result[j-i]`. Od kodu k4 różni go tylko proces zapisu wyniku, mający na celu zniwelowanie strat wydajności wywołanych wyścigiem, i dużej ilości kolizji w pamięci cache. Podejście to zapobiega wyścigowi, jednak nie rozwiązuje w pełni problemu **false sharingu**, ponieważ wątki wciąż mogą zapisywać do tych samych linii chache (tyllko tym razem już nie do tych samych elementów tablicy) .....

```
int calculate(int n, int m){
        //...alokacja pamieci i zliczanie PrimeArray jak wczesniej

        int limit = sqrt(n);
        #pragma omp parallel for schedule(dynamic, 1)
        for (int i = 2; i <= limit; i++){
                if (primeArray[i]) {
                        int firstMultiple = (m / i);
                        if (firstMultiple <= 1) {
                                firstMultiple = i + i;
                        } else if (m % i){
                                firstMultiple = (firstMultiple * i) + i;
                        } else {
                                firstMultiple = (firstMultiple * i);
                        }
                        for (int j = firstMultiple; j <= n; j+=i) {
                                if (result[j-m]) result[j-m] = false;
                        }
                }
        }

        //zliczanie prime_counter i zwalnianie pamieci jak w k4
}
```

Listing 6: Kod programu k4a.cpp

K5 - Sito równoległe domenowe z lokalnością dostępu do danych

Oznaczenie skrótowe: PAR-SIEVE-DOM-LL

Uzasadnienie wyboru rozmiaru bloku: Tak jak już zostało napisane w wypadku kodu k3, blok został wyznaczony w ramach przeprowadzonego eksperymentu wpływu rozmiarów na prędkość wykonywania kodu.

Uzasadnienie wyboru podziału pracy: W tym wypadku została wybrana strategia `(static)`, dzięki podziałowi na bloki, obciążenie czasowe poszczególnych zadań jest wyrównane (nierówność występuje tylko na $\sqrt{j}$), co sprawia, że nie potrzebujemy użycia metod dynamicznych. Uniknięcie strategii dynamicznych pozwala nam uniknąć dodatkowego narzutu komunikacyjnego.

```
int calculate(m, n){
        //...alokacja pamieci i liczenie PrimeArray jak wczesniej

        int blockSize = 65536;
        int numberOfBlocks = (n - m) / blockSize + ((n - m) % blockSize != 0);

        #pragma omp parallel for schedule(static)
        for (int i = 0; i < numberOfBlocks; i++) {
                int low = m + i * blockSize;
                int high = low + blockSize - 1;
                if (high > n) high = n;

                memset(&result[low - m], true, (high - low + 1) * sizeof(bool));

                for (int j = 2; j * j <= high; j++) {
                        if (primeArray[j]) {
                                int firstMultiple = (low / j) * j;
                                if (firstMultiple < low) firstMultiple += j;
                                if (firstMultiple == j) firstMultiple += j;

                                for (int k = firstMultiple; k <= high; k += j) {
                                        result[k - m] = false;
                                }
                        }
                }
        }

        //zliczanie prime_counter i zwalnianie pamieci jak wczesniej
}
```

Listing 7: Kod programu k5.cpp

3 Prezentacja wyników i omówienie przebiegu eksperymentu.

Zasady pracy z oprogramowaniem VTune.

Porównanie podstawowych parametrów wywołania dla kodów k1-k5.

Porównania zostały wykonane dla kodów przedstawionych w poprzedniej sekcji, z tymi samymi schedulingami i rozmiarami bloków na zakresie liczb od 1 do 100 milionów.

	k1	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Elapsed Time [s]	23,750		0.660	0.130	0.381		
Prędkość przetwarzania [mln l. pierwszych/s]	4.210		14.394	35.714	15.121		
Instructions retired [mln]	405,744		0,515	0,777	2,179		
Clockticks [bilionów tików]	105,112		2,914	0,566	19,808		
Effective PCU	7.7%		4.7%	7.1%	42.4%		
Badania kodów równoległych	**k1**	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Przyspieszenie względem k3a	-		-	-	0.34%		
Efektywność [Acc/Cores]	-		-	-	0.028		

Tablica 2: Porównanie podstawowych parametrów wywołania dla kodów k1-k5.

Performance Cores	**k1**	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Retiring	66.0%		1.2%	0.0%	1.1%		
Front End Bound	11.4%		0.0%	3.1%	3.6%		
Back End Bound	3.3%		100.0%	0.0%	78.7%		
Memory Bound	0.2%		100.0%	0.0%	67.4%		
Core Bound	3.1%		0.0%	0.0%	11.4%		
Efficient Cores	**k1**	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Retiring	0.0%		0.0%	0.0%	3.0%		
Front End Bound	0.0%		0.0%	0.0%	1.3%		
Back End Bound	4.1%		0.0%	0.0%	90.8%		
Memory Bound	0.0%		0.0%	0.0%	53.0%		
Core Bound	4.1%		0.0%	0.0%	37.8%		

Tablica 3: Wyniki analizy przyczyn ograniczeń wydajności dla kodów k1-k5 z podziałem na Performance i Efficient Cores.

Porównanie działań kodów dla różnych zakresów liczb.

W ramach tego eksperymentu zmierzyliśmy Ellapsed Time dla kodów k1-k5 dla trzech różnych zakresów liczb: <2; max>, <max/2; max> oraz <2; max/2>, gdzie `max` to 10^8.

	k1	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
<2; max>							
<max/2; max>							
<2; max/2>							

Tablica 4: Porównanie prędkości działania dla różnych zakresów liczb w kodach k1-k5.

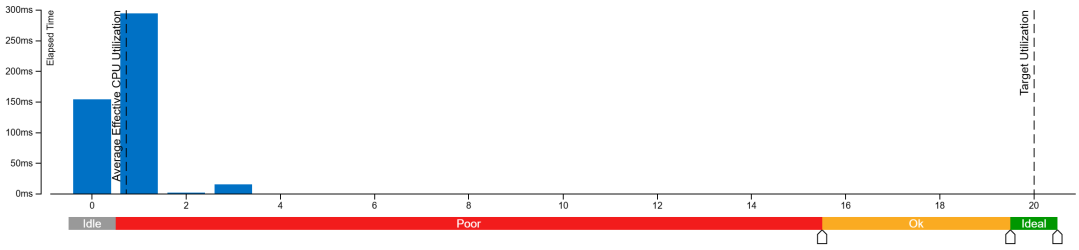
Badanie wpływu różnych szeregowań dla czasu przetwarzania kodów k2, k4, k4a i k5.

Porównanie parametrów przetwarzania kodów k3 i k3a.

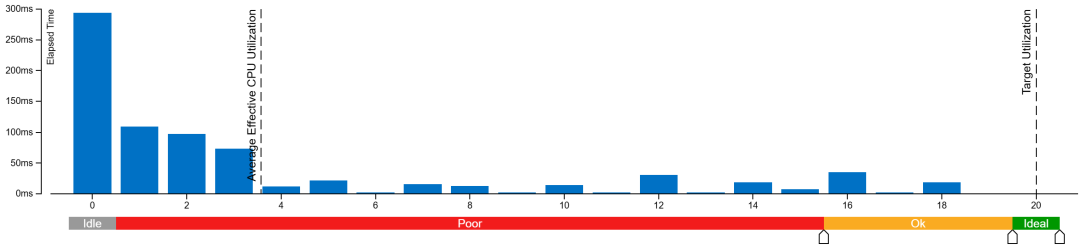
Wyniki eksperymentu wyznaczania wielkości bloku dla kodów k3a i k5.

Eksperyment - zwiększenie zakresu działania kodu k5 do 1 miliarda.

W czasie przeprowadzania eksperymentu k5 zauważyliśmy w teście threading niespodziewanie niskie CPU utilization, co skłoniło nas do przeprowadzenia kolejnego eksperymentu. Postanowiliśmy przeprowadzić dodatkowe badanie tym razem dla zakresu liczby zwiększonego o 1 potęgę 10 (więcej już by się nie dało ze względu na ograniczenia pamięciowe). Poniżej zamieściliśmy wykresy z porównaniem histogramów CPU utilization dla zakres 1exp8 i 1exp9.



Rysunek 1: CPU utilization dla kodu 5 dla zakresu 1exp8.



Rysunek 2: CPU utilization dla kodu 5 dla zakresu 1exp9.

Dzięki temu porównaniu widzimy, że kod nie działa źle, czas w stanie 0/1 jest stały i jest to czas spędzony w sekcji sekwencyjnej (liczenie małego sita). Jest to dla obu uruchomień około 0.3 sekundy. Zmienia się między nimi zaś czas spędzony w sekcji równoległej (0.158 dla 1exp8 i 0.404 dla 1exp9). Wraz ze zwiększeniem się tego czasu możemy też zaobserwować wzort czasu w którym pracuje wiele wątków (co oznacza, że dobrze dzielą się zadaniami, mimo zwiększenia czasu w części równoległej nie zwiększyła się sumaryczna liczba czasu w stanie 0/1)

4 Wnioski.